

VFG 경로추종 시스템 — 배경이론 보고서

Vector Field Guidance Path Following: Background Theory

이수원
국민대학교 기계공학부

2026년 3월

Abstract

이 보고서는 LIMO 모바일 로봇을 위한 VFG(Vector Field Guidance) 기반 경로추종 시스템의 배경이론을 학부생 수준에서 설명한다. 경로추종 문제 정의, VFG 유도 원리, PID 및 LPV $\mathcal{H}_\infty$ 제어기 개념, 시뮬레이션 결과 비교를 다룬다. $\mathcal{H}_\infty$ 합성의 수학적 세부사항은 생략하고, 학생이 시뮬레이션 도구를 활용하여 경로추종 실험을 수행하는 데 필요한 직관적 이해를 제공하는 것을 목표로 한다.

Contents

1	경로추종 문제 정의	2
1.1	경로(Path)란?	2
1.2	추종 오차	2
1.3	자전거 모델 (Bicycle Model)	2
2	VFG 유도 원리	2
2.1	Vector Field Guidance란?	2
2.2	수렴/추종 벡터 분해	3
2.3	$\tanh$ 가중 함수	3
2.4	원하는 방향각	3
2.5	VFG의 역할 구분	3
3	제어기 개념	3
3.1	PID + 곡률 피드포워드 (PID-FF)	3
3.2	LPV $\mathcal{H}_\infty$ 제어기 (블랙박스 관점)	4
3.2.1	$\mathcal{H}_\infty$ 제어란?	4
3.2.2	LPV 스케줄링	4
3.2.3	입출력 구조	4
3.3	두 제어기의 비교	4
4	시뮬레이션 결과 비교	5
4.1	실험 설정	5
4.2	속도별 성능	5
4.3	주요 관찰	5
4.4	시뮬레이션 실행 방법	5
4.5	정리	6

1 경로추종 문제 정의

1.1 경로(Path)란?

경로추종(path following)에서 **경로**란 로봇이 따라가야 할 2차원 기하학적 곡선이다. 시간에 대한 정보는 포함하지 않으며, 호 길이(arc-length) s 로 매개변수화한다:

$$\mathbf{P}(s) = \begin{bmatrix} x(s) \\ y(s) \end{bmatrix}, \quad s \in [0, L] \quad (1)$$

여기서 L 은 경로의 전체 길이이다.

경로 위의 각 점에서 다음 기하 정보를 정의할 수 있다:

- 접선 벡터 $\mathbf{t}(s)$: 경로 진행 방향의 단위 벡터
- 법선 벡터 $\mathbf{n}(s)$: 접선에 수직, 왼쪽 방향 (반시계)
- 곡률 $\kappa(s)$: 경로의 “휘어진 정도”. $\kappa = 1/R$ (원호의 경우 반지름의 역수)
- 경로 방향각 $\psi_{\text{path}}(s) = \text{atan2}(y'(s), x'(s))$

1.2 추종 오차

로봇 위치 $\mathbf{q} = (X, Y)$ 에서 경로 위 가장 가까운 점 $\mathbf{P}(s^*)$ 를 찾으면, 두 가지 오차를 정의할 수 있다:

- 횡방향 오차 (cross-track error): $e_d = (\mathbf{q} - \mathbf{P}(s^*)) \cdot \mathbf{n}(s^*)$
- 방향 오차 (heading error): $e_\psi = \psi_{\text{des}} - \psi$

$e_d > 0$ 이면 로봇이 경로 왼쪽에, $e_d < 0$ 이면 오른쪽에 있다. $e_\psi > 0$ 이면 로봇이 원하는 방향보다 시계 방향으로 벗어나 있다.

경로추종의 목표는 두 오차를 모두 0으로 수렴시키는 것이다:

$$e_d \rightarrow 0, \quad e_\psi \rightarrow 0 \quad (2)$$

1.3 자전거 모델 (Bicycle Model)

LIMO 로봇은 Ackermann 조향 구조를 갖는 1:10 스케일카이다. 저속에서는 운동학적 자전거 모델(kinematic bicycle model)로 충분히 근사할 수 있다.

$$\begin{aligned} \dot{X} &= v \cos(\psi + \beta) \\ \dot{Y} &= v \sin(\psi + \beta) \\ \dot{\psi} &= \frac{v}{L} \cos \beta \cdot \tan \delta \\ \dot{\delta} &= \frac{\delta_{\text{cmd}} - \delta}{\tau_\delta} \end{aligned} \quad (3)$$

여기서 v : 종방향 속도, ψ : 방향각, δ : 조향각, $L = 0.2\text{m}$: 축거, $\tau_\delta = 0.07\text{s}$: 조향 시정수, β : 슬립각이다.

핵심 매개변수:

- 축거 $L = 0.2\text{m}$ — 앞바퀴와 뒷바퀴 축 사이 거리
- 조향 시정수 $\tau_\delta = 0.07\text{s}$ — 조향 명령이 실제 조향각에 반영되는 지연
- 최대 조향각 $\delta_{\text{max}} = 0.5\text{rad}$ ($\approx 28.6^\circ$)

조향 명령 δ_{cmd} 를 입력하면, 1차 지연을 거쳐 실제 조향각 δ 에 반영된다. 이 조향각이 차량의 방향을 변화시키고, 결과적으로 경로추종 오차를 줄인다.

2 VFG 유도 원리

2.1 Vector Field Guidance란?

VFG(Vector Field Guidance)는 경로 주변에 “흐름의 장”을 형성하여 로봇이 자연스럽게 경로로 수렴하고 경로를 따라가도록 유도하는 기법이다.

직관적으로, 강물의 흐름을 상상하면 된다:

- 경로에서 멀리 있으면 → 강하게 경로 쪽으로 당기는 흐름 (수렴)
 - 경로 위에 있으면 → 경로 방향으로 흘러가는 흐름 (추종)
- 이 두 성분의 자연스러운 전환이 VFG의 핵심이다.

2.2 수렴/추종 벡터 분해

경로 위 가장 가까운 점 $\mathbf{P}(s^*)$ 에서 두 방향 벡터를 정의한다:

- 추종 벡터 $\mathbf{v}_{\text{trav}} = \mathbf{t}(s^*)$: 경로 접선 방향
 - 수렴 벡터 $\mathbf{v}_{\text{conv}} = -\text{sgn}(e_d) \cdot \mathbf{n}(s^*)$: 경로를 향하는 방향
- 이 두 벡터를 거리에 따라 가중 합산한다:

$$\mathbf{v}_{\text{des}} = k_{\text{conv}}(d) \cdot \mathbf{v}_{\text{conv}} + k_{\text{trav}}(d) \cdot \mathbf{v}_{\text{trav}} \quad (4)$$

여기서 $d = |e_d|$ 는 경로까지의 거리이다.

2.3 tanh 가중 함수

가중 계수는 tanh 함수로 정의한다:

$$k_{\text{conv}}(d) = \tanh(k_e \cdot d), \quad k_{\text{trav}}(d) = \sqrt{1 - k_{\text{conv}}^2(d)} \quad (5)$$

k_e 는 수렴 이득(convergence gain)으로, 경로로 수렴하는 강도를 결정한다.

- $d \gg 0$: $\tanh(k_e d) \approx 1 \rightarrow$ 수렴 우세 (경로 쪽으로 이동)
 - $d \approx 0$: $\tanh(k_e d) \approx 0 \rightarrow$ 추종 우세 (경로를 따라 이동)
 - k_e 증가: 더 빠르게 수렴, 그러나 너무 크면 오버슈트 발생
- 기본값 $k_e = 3.0$ 은 대부분의 상황에서 적절한 수렴 성능을 보인다.

2.4 원하는 방향각

VFG의 출력은 원하는 방향각(desired heading)이다:

$$\psi_{\text{des}} = \text{atan2}(v_{\text{des},y}, v_{\text{des},x}) \quad (6)$$

이 ψ_{des} 와 실제 로봇 방향각 ψ 의 차이가 방향 오차 e_ψ 이며, 제어기가 이 오차를 줄이도록 조향 명령을 생성한다.

2.5 VFG의 역할 구분

VFG는 “어디로 향해야 하는가”를 결정하고 (유도, guidance), 제어기는 “조향을 얼마나 할 것인가”를 결정한다 (제어, control).

이 분리 구조 덕분에:

- VFG는 경로 기하에만 의존 → 제어기 교체 시에도 유도 모듈 재사용 가능
- 제어기는 방향 오차만 처리 → 다양한 제어 기법 적용 가능
- 공정한 비교: 같은 VFG 위에서 PID와 LPV $\mathcal{H}_\infty$ 를 직접 비교할 수 있음

3 제어기 개념

3.1 PID + 곡률 피드포워드 (PID-FF)

가장 직관적인 제어기이다. 방향 오차 $e_\psi = \psi_{\text{des}} - \psi$ 에 비례-미분 제어를 적용하고, 경로 곡률에 대한 피드포워드 항을 추가한다:

$$\delta_{\text{cmd}} = \underbrace{K_P \cdot e_\psi + K_D \cdot \dot{e}_\psi}_{\text{PID 피드백}} + \underbrace{\arctan(L \cdot \kappa)}_{\text{곡률 피드포워드}} \quad (7)$$

- $K_P = 2.0$: 비례 이득. 오차에 비례하여 조향
- $K_D = 0.3$: 미분 이득. 오차 변화율에 반응하여 진동 억제
- 피드포워드: 경로가 휘어져 있으면 그에 맞는 조향을 미리 적용

PID-FF는 구현이 간단하고 직관적이지만, 속도와 곡률이 변하는 상황에서 이득이 고정되어 있어 최적 성능을 보장하지 못한다.

3.2 LPV $\mathcal{H}_\infty$ 제어기 (블랙박스 관점)

3.2.1 $\mathcal{H}_\infty$ 제어란?

$\mathcal{H}_\infty$ 제어는 “최악의 경우에도 성능을 보장”하는 강인 제어 기법이다. 일상적인 비유: 강인한 자동조종 장치.

- 센서 잡음이 있어도 안정적으로 동작
- 모델과 실제 로봇 사이 차이가 있어도 강인
- 주파수 영역에서 성능/강인성 트레이드오프를 명시적으로 설계

$\mathcal{H}_\infty$ 합성의 수학적 세부 사항은 이 보고서의 범위를 벗어난다. 학생은 사전 계산된 제어기 (JSON 파일)를 그대로 사용하면 된다.

3.2.2 LPV 스케줄링

LPV(Linear Parameter-Varying)는 “상황에 따라 이득이 자동으로 변하는” 제어기이다.

스케줄링 매개변수 ρ 를 다음과 같이 정의한다:

$$\rho = |\kappa| \cdot v \quad (8)$$

ρ 의 물리적 의미:

- $\rho = 0$: 직선 구간 또는 정지 $\rightarrow$ 제어기가 보수적으로 동작
- ρ 큼: 고속 + 고곡률 $\rightarrow$ 제어기가 적극적으로 반응

6개의 꼭짓점(vertex) 제어기 ($\rho = 0, 1, 2, 3, 4, 5$)가 사전 설계되어 있으며, 실시간에서 현재 ρ 값에 따라 인접 꼭짓점 사이를 선형 보간한다.

3.2.3 입출력 구조

학생이 알아야 할 것은 입출력 관계뿐이다:

- 입력: 방향 오차 e_ψ , 측정 조향각 δ_{meas} , 스케줄링 변수 ρ
- 출력: 조향 명령 δ_{cmd}
- 내부 상태: 6차 상태공간 모델 (자동 관리)

부호 규칙: 학생은 $e_\psi = \psi_{\text{des}} - \psi$ 를 그대로 넘기면 된다. 내부적으로 MATLAB 관례에 맞게 부호가 자동 변환된다.

3.3 두 제어기의 비교

Table 1: PID-FF와 LPV $\mathcal{H}_\infty$ 비교

항목	PID-FF	LPV $\mathcal{H}_\infty$
구현 난이도	쉬움	블랙박스 (JSON 로드)
이득 조정	수동 (K_P, K_D)	자동 (ρ 기반)
잡음 강인성	보통	높음 (설계에 반영)
고속 성능	고정 이득 한계	ρ 스케줄링으로 적응
저속 직선	우수	유사
파라미터 수	2 (K_P, K_D)	0 (사전 설계)

핵심 메시지: 저속 직선 구간에서는 PID로 충분하지만, 고속이나 급곡선에서는 LPV $\mathcal{H}_\infty$ 가 체계적인 이점을 보인다. 다음 장의 시뮬레이션 결과에서 이를 정량적으로 확인한다.

4 시뮬레이션 결과 비교

4.1 실험 설정

시뮬레이션은 `vfg_pathfollowing` 패키지를 사용하여 수행한다. 동일한 VFG 유도 모듈($k_e = 3.0$) 위에서 PID-FF와 LPV $\mathcal{H}_\infty$ 를 비교한다.

경로: 직선 $\rightarrow$ 원호($R=0.5\text{ m}$, $\theta = 90^\circ$) $\rightarrow$ 직선 (step curvature path)

이 경로는 갑작스러운 곡률 변화를 포함하여, 제어기의 과도 응답 성능을 평가하기에 적합하다.

4.2 속도별 성능

표 2에 속도별 RMS 방향 오차와 최대 방향 오차를 정리하였다.

Table 2: 속도별 방향 오차 비교 (Step curvature path, $R=0.5\text{ m}$)

v [m/s]	RMS e_ψ [deg]		Max e_ψ [deg]		
	PID-FF	LPV $\mathcal{H}_\infty$	PID-FF	LPV $\mathcal{H}_\infty$	
1.0	4.17	2.83	15.8	14.2	※ 200회 Monte Carlo 시뮬레이션 (센서
1.5	3.77	2.34	18.1	11.5	
2.0	3.40	2.16	19.4	8.7	
2.5	3.39	2.14	20.2	7.3	

잡음 포함) 평균값

4.3 주요 관찰

1. **저속 ($v = 1.0\text{ m/s}$):** 두 제어기 모두 양호한 성능. PID와 LPV의 차이가 작다. 이 영역에서 $\rho = |\kappa| \cdot v$ 가 작아 LPV 스케줄링의 이점이 제한적이다.
2. **중속 ($v = 1.5\text{ m/s}$):** LPV가 RMS에서 38% 우위를 보이기 시작한다. 곡률 구간 진입 시 ρ 가 증가하면서 제어기 대역폭이 자동으로 확대된다.
3. **고속 ($v \geq 2.0\text{ m/s}$):** LPV의 우위가 뚜렷하다. 특히 **최대 오차**에서 큰 차이가 난다 — $v = 2.0$ 에서 PID 19.4° vs LPV 8.7° (55% 감소). PID는 고정 이득이므로 속도가 증가해도 동일한 이득을 사용하지만, LPV는 ρ 증가에 따라 대역폭을 자동으로 높여 더 빠르게 반응한다.
4. **센서 잡음 영향:** 두 제어기 모두 잡음($\sigma_\psi = 0.03\text{ rad}$) 하에서 안정적이다. LPV는 $\mathcal{H}_\infty$ 설계 시 잡음 모델이 포함되어 있어 추가적인 강인성을 보인다.

4.4 시뮬레이션 실행 방법

노트북 02_pid_vs_lpvhinf.ipynb에서 위 결과를 직접 재현할 수 있다:

```
from vfg_pathfollowing import Simulator, StepCurvaturePath

path = StepCurvaturePath(R=0.5, theta_arc=1.57)
sim = Simulator(path, speed=2.0)
results = sim.compare(T=15.0)
Simulator.plot_comparison(results, path=path)
```

4.5 정리

VFG 유도 + LPV $\mathcal{H}_\infty$ 제어 조합은 PID-FF 대비 고속/고곡률 구간에서 체계적인 성능 향상을 보인다. 이는 ρ -스케줄링을 통해 주행 조건에 맞게 제어기 대역폭을 자동 조절하기 때문이다. 학생은 노트북을 통해 다양한 속도, 경로, 파라미터에서 이 차이를 직접 확인할 수 있다.